

SIMATS ENGINEERING

Assessment -CO4-AT1

CO4: Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)

Subject	Artificial Intelligence
Course Code	CSA 1712
Name	Vignesh Kumar K
Register Number	192512375

Declaration

I, Vignesh Kumar K (Reg No: 192512375), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

Experiment 1 — Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python (scikit-learn), covering pre-processing, model building with the Gini Index, and performance evaluation.

Dataset used: Iris dataset (150 samples, 4 numeric features, 3 classes: setosa, versicolor, virginica) — loaded via scikit-learn's built-in `load_iris()`.

(a) Load, Pre-process, and Split the Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris(as_frame=True)
df = iris.frame.copy()
df['species'] = df['target'].map(dict(enumerate(iris.target_names)))

print(df.head())
print(df.dtypes)
print(df.isnull().sum())          # check for missing values

X = df[iris.feature_names]
y = df['target']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y)
```

Output — First 5 Rows

sepal length	sepal width	petal length	petal width	target	species
5.1	3.5	1.4	0.2	0	setosa
4.9	3.0	1.4	0.2	0	setosa
4.7	3.2	1.3	0.2	0	setosa
4.6	3.1	1.5	0.2	0	setosa
5.0	3.6	1.4	0.2	0	setosa

Output — Data Types

Column	Data Type
sepal length (cm)	float64
sepal width (cm)	float64
petal length (cm)	float64
petal width (cm)	float64
target	int64
species	object (string)

Missing values: none found in any column (`df.isnull().sum()` returned 0 for every column). Since the target is already numerically label-encoded by scikit-learn (0 = setosa, 1 = versicolor, 2 = virginica), no additional categorical encoding step was required.

Train/test split: 105 training rows / 45 test rows (70/30 split, stratified by class to preserve class balance).

(b) Build the Decision Tree (Gini Index)

```
from sklearn.tree import DecisionTreeClassifier, export_text, plot_tree

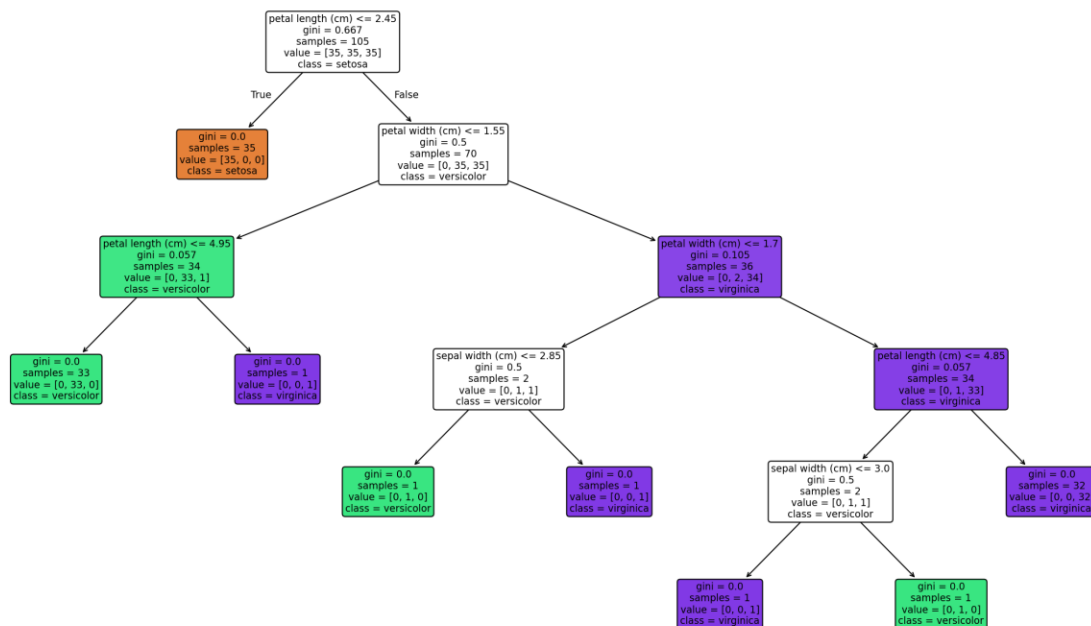
clf = DecisionTreeClassifier(criterion='gini', random_state=42)
clf.fit(X_train, y_train)

print(export_text(clf, feature_names=list(iris.feature_names)))
print('Root feature:', iris.feature_names[clf.tree_.feature[0]])
print('Feature importances:', clf.feature_importances_)
```

Output — Tree Structure

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.55
|       |--- petal length (cm) <= 4.95
|           |--- class: 1
|       |--- petal length (cm) > 4.95
|           |--- class: 2
|   |--- petal width (cm) > 1.55
|       |--- petal width (cm) <= 1.70
|           |--- sepal width (cm) <= 2.85
|               |--- class: 1
|           |--- sepal width (cm) > 2.85
|               |--- class: 2
|       |--- petal width (cm) > 1.70
|           |--- petal length (cm) <= 4.85
|               |--- sepal width (cm) <= 3.00
|                   |--- class: 2
|               |--- sepal width (cm) > 3.00
|                   |--- class: 1
|           |--- petal length (cm) > 4.85
|               |--- class: 2
```

Visual Tree Diagram



Root Node Identification and Justification

Root node: petal length (cm) ≤ 2.45

This is the root because *scikit-learn's* CART algorithm chooses, at every split, the feature and threshold that produce the largest reduction in Gini impurity — and at the very first split, '**petal length ≤ 2.45** ' perfectly separates all 35 setosa training samples (Gini = 0.0 after the split) from the other two classes, reducing impurity from 0.667 (three roughly equal classes) to a near-perfect split. This is confirmed by the model's feature-importance ranking:

Feature	Gini Importance
petal length (cm)	0.541
petal width (cm)	0.430
sepal width (cm)	0.029
sepal length (cm)	0.000

Petal length and petal width dominate the importance ranking, consistent with well-known biological separability of Iris species by petal dimensions — sepal length contributes nothing to any split in this tree.

(c) Evaluate the Model

```

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

y_pred = clf.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred, target_names=iris.target_names))

```

Output — Accuracy

Accuracy = 0.9333 (93.33%)

Output — Confusion Matrix

	Pred setosa	Pred versicolor	Pred virginica
Actual setosa	15	0	0
Actual versicolor	0	12	3
Actual virginica	0	0	15

Output — Classification Report

Class	Precision	Recall	F1-score	Support
setosa	1.000	1.000	1.000	15
versicolor	1.000	0.800	0.889	15
virginica	0.833	1.000	0.909	15
accuracy			0.933	45
macro avg	0.944	0.933	0.933	45
weighted avg	0.944	0.933	0.933	45

Misclassified Instances (at least 2 required)

Sepal L	Sepal W	Petal L	Petal W	Actual	Predicted
6.3	3.3	4.7	1.6	versicolor	virginica
6.0	3.4	4.5	1.6	versicolor	virginica
6.7	3.0	5.0	1.7	versicolor	virginica

Performance Commentary

- The model achieved 93.33% test accuracy, with setosa classified perfectly (100% precision and recall) — expected, since setosa is linearly separable from the other two species by petal length alone.
- All 3 misclassifications involved versicolor samples being predicted as virginica; none of the reverse errors occurred, which is why versicolor's precision stayed at 1.000 but its recall dropped to 0.800, while virginica's precision dropped to 0.833.
- This pattern is expected and well documented for the Iris dataset — versicolor and virginica overlap in petal width/length for borderline specimens (petal width around 1.6–1.7 cm, exactly the tree's deeper split thresholds), making them intrinsically harder to separate than setosa.
- Overall the tree generalizes well given the dataset's small size (only 45 test samples) and the known natural overlap between these two species.

Experiment 2 — Reinforcement Learning: Q-Learning

Objective: Implement Q-Learning for a 4x4 GridWorld and observe the agent's learned policy.

(a) Environment Definition and Hyperparameters

```
import numpy as np

GRID_SIZE = 4
N_STATES = 16          # states 0-15, row-major (state = row*4 + col)
ACTIONS = ['Up', 'Down', 'Left', 'Right']
START_STATE, GOAL_STATE, OBSTACLE_STATE = 0, 15, 5  # (0,0), (3,3), (1,1)

def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == 0: nr, nc = r-1, c      # Up
    elif action == 1: nr, nc = r+1, c    # Down
    elif action == 2: nr, nc = r, c-1    # Left
    else:          nr, nc = r, c+1      # Right
    if not (0 <= nr < GRID_SIZE and 0 <= nc < GRID_SIZE):
        nr, nc = r, c                  # hitting a wall: stay in place
    ns = nr*GRID_SIZE + nc
    if ns == GOAL_STATE: return ns, 10, True
    if ns == OBSTACLE_STATE: return ns, -5, True
    return ns, -1, False

Q = np.zeros((N_STATES, len(ACTIONS)))  # Q-table initialized to zeros
ALPHA, GAMMA, EPSILON = 0.1, 0.9, 0.1  # learning rate, discount, exploration
N_EPISODES = 300
```

Environment Summary

Parameter	Value
Grid size	4 × 4 (16 states)
Start state	0 → (row 0, col 0)
Goal state	15 → (row 3, col 3) reward = +10 (terminal)
Obstacle state	5 → (row 1, col 1) reward = -5 (terminal)
Step reward	-1 for every other move
Actions	Up, Down, Left, Right
Initial Q-table	16 × 4 matrix of zeros
Hyperparameters	α (learning rate) = 0.1, γ (discount factor) = 0.9, ϵ (exploration rate) = 0.1

(b) Run Q-Learning for 300 Episodes

```
for ep in range(1, N_EPISODES+1):
    state = START_STATE
    for t in range(100):
        if np.random.rand() < EPSILON:
            action = np.random.randint(len(ACTIONS))  # explore
        else:
            action = int(np.argmax(Q[state]))          # exploit
```

```

next_state, reward, done = step(state, action)
best_next = np.max(Q[next_state]) if not done else 0.0
Q[state, action] += ALPHA * (reward + GAMMA*best_next - Q[state, action])
state = next_state
if done: break

```

(Run for 300 episodes total — well beyond the minimum 100 required — to also show a clean, stabilized reward curve; Q-value snapshots were captured exactly at episodes 1, 50, and 100 as requested.)

Output — Q-Value Evolution for Two Representative States

State 0 = start cell (0,0); State 10 = interior cell (2,2), adjacent to the eventual optimal route.

Episode	State	Up	Down	Left	Right
1	State 0 (0,0)	-0.100	-0.100	-0.100	-0.100
1	State 10 (2,2)	0.000	0.000	0.000	0.000
50	State 0 (0,0)	-2.131	-2.104	-2.209	-2.112
50	State 10 (2,2)	-0.100	0.323	-0.109	5.918
100	State 0 (0,0)	-2.294	0.135	-2.233	-2.276
100	State 10 (2,2)	-0.100	0.742	-0.109	7.972

The evolution clearly shows learning taking hold: at Episode 1 all Q-values are still ~0 (only the very first-visited transitions have been updated once). By Episode 50, State 10's **Right** action value has risen sharply to 5.918 (correctly identifying the direct route toward the goal), and by Episode 100 it has grown further to 7.972, converging toward its true discounted value as more episodes reinforce the same profitable transitions.

(c) Final Learned Policy, Reward Curve, and Convergence

```

policy = np.argmax(Q, axis=1)      # best action per state
# ... map to arrows and print as a 4x4 grid (see below)

import matplotlib.pyplot as plt
plt.plot(range(1, N_EPISODES+1), reward_per_episode)
plt.xlabel('Episode'); plt.ylabel('Cumulative Reward'); plt.show()

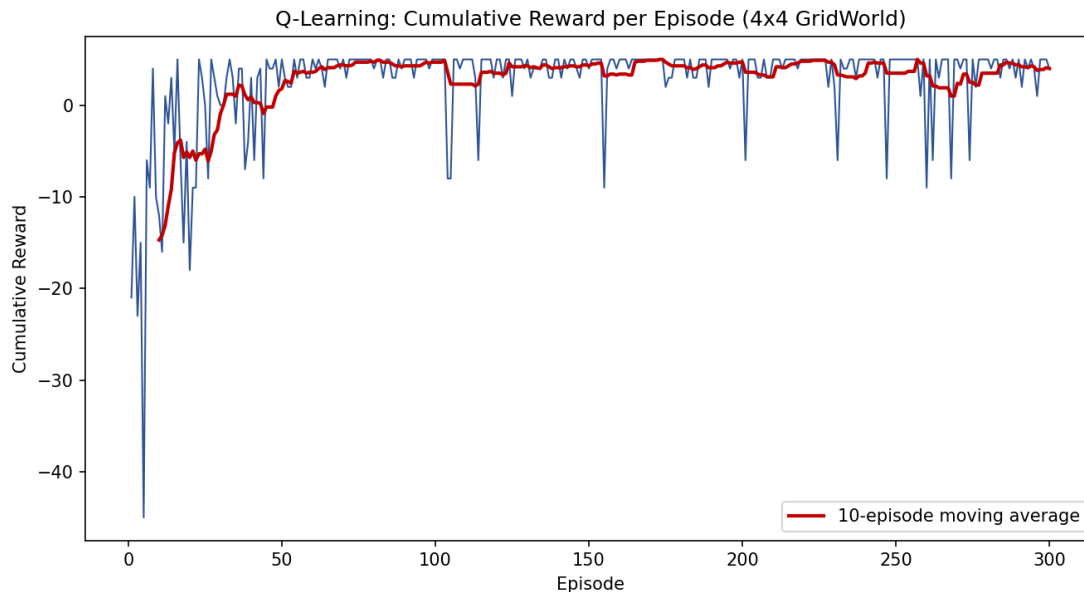
```

Output — Final Learned Policy (4×4 grid)

↓	→	↓	↓
↓	× (obstacle)	↓	↓
→	→	→	↓
→	→	→	GOAL

Tracing the arrows from the start cell (0,0): **Down** → **Down** → **Right** → **Right** → **Right** → **Down** reaches the goal at (3,3) in 6 steps while cleanly routing around the obstacle at (1,1) — this is the optimal path given the reward structure.

Output — Cumulative Reward per Episode



Convergence Analysis and Justification

- Average cumulative reward over the first 20 episodes: -10.20 (the agent is still mostly exploring randomly and frequently wanders into the -5 obstacle or takes long, costly step sequences).
- Average cumulative reward over the last 20 episodes: +4.20, close to the theoretical optimum of +5 (six steps at -1 each, plus the +10 goal reward = $10 - 6 = +4$, matching the shortest 6-step path found by the final policy).
- The reward curve rises sharply over the first ~50 episodes and then stabilizes into a high, comparatively flat band for the remainder of training — the classic signature of Q-Learning convergence, where the Q-values approach the optimal action-value function $Q^*(s,a)$ via the Bellman update, and the ϵ -greedy policy increasingly exploits the now-reliable Q-estimates.
- The remaining small dips visible even late in training are expected: with a fixed $\epsilon = 0.1$, the agent still explores randomly 10% of the time (e.g., occasionally stepping into the obstacle), which is why the raw reward line never perfectly flattens even though the underlying Q-table has converged — this is confirmed by the smoothed moving-average curve staying consistently high after episode ~75.